PROJECT AND TEAM INFORMATION

Project Title

TradeX: A Real-Time Stock Order Book and Matching System

Student / Team Information

Team Name: *Team DSCPP-III-26-T029*	*Order Book Operators*
Team member 1 (Team Lead)	*Aggarwal, Priyanshi – 2510010617* *priyanshiaggarwal1812@gmail.com* 
Team member 2	*Mehara, Shivani – 2510012532* *shivanimehara1901@gmail.com* 
Team member 3	*Bisht, Aastha – 2510012019* *aasthabisht2008@gmail.com* 
Team member 4	*Negi, Khushi – 2510012249* *khushinegi4455@gmail.com* 

PROPOSAL DESCRIPTION

Motivation

Financial markets handle massive buy/sell orders that require highly efficient processing. A poorly designed system causes slow matching and resource waste. This project designs an Order Book and Trading Engine simulating a real stock exchange using price-time priority matching logic. It applies fundamental Data Structures and Algorithms (linked lists, hash tables, trees, and sorting) alongside C++ Object-Oriented Programming (OOP) to model orders, portfolios, and matching engines. To maximize speed, the engine avoids heavy databases, running purely in-memory (RAM) with a flat-file verification gate (users.txt) to guarantee low-latency performance.

State of the Art / Current solution

Industrial electronic exchanges deploy pure in-memory architectures to maintain active books inside volatile memory (RAM), eliminating disk I/O bottlenecks. In contrast, standard designs often rely on relational databases or linear search loops ($O(N)$) that degrade under high volumes. This project implements a modular C++ framework integrating self-sorting trees with hash indexing. This design eliminates sequential scanning delays, resolving transaction crossing logic instantly within memory.

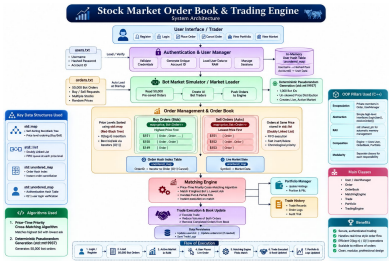
Project Goals and Milestones

The system achieves high performance and security through eight distinct goals: (1) authenticating sessions via `users.txt` *with tracking IDs, (2) running active transaction books inside RAM to bypass disk lag, and (3) maintaining a cold log database via* `orders.txt`*. Additionally, it (4) initializes 50,000 algorithmic bot records at startup, (5) implements price-time priority sorting, and (6) processes instant order cancellations in constant $O(1)$ complexity. Finally, the core engine (7) automatically executes intersecting crossed trades and (8) renders real-time side-by-side Market Depth grids and interactive Trader Profile summaries for the authenticated client on the terminal console.*

Project Approach

- *Developed on Windows using VS Code with the MinGW-w64 (GCC/g++) toolchain, using flat text files (*`users.txt` *and* `orders.txt`*) for cold data backups.*
- *Runs an In-Memory Architecture inside RAM to eliminate traditional hard drive and database file processing delays.*
- *Deploys C++ STL containers (*`std::map`*,* `std::list`*,* `std::unordered_map`*) to handle self-sorting price lines, FIFO time queues, and instant $O(1)$ order cancellations.*

System Architecture (High Level Diagram)



Project Outcome / Deliverables

This project builds a fast C++ trading program split into a `UserManager`, `OrderBook`, *and matching* `Engine` *without slow database files. Users log in securely via a text file (*`users.txt`*) in instant* $O(1)$ *time. Once authorized, the engine loads 50,000 background bot orders directly into RAM using strict price-time priority rules. A quick hash map index allows users to cancel pending orders instantly in* $O(1)$ *time. Finally, the terminal screen lets you see live side-by-side* ***Market Depth*** *and your active personal* ***Trader Profile*** *updates on the fly.*

Assumptions

- *Single-Threaded Pipeline: The matching engine processes incoming operations sequentially to ensure safe, race-free in-memory execution.*
- *Single-Asset Context: Core computations handle a single stock ticker without currency cross-conversion tracks.*
- *Standardized Transactions: Ingestion parameters are restricted entirely to standard Limit and Market orders.*
- *Pre-Funded Balances: Accounts are assumed pre-funded, maximizing engine throughput speed.*

References

1 *R. Cont, S. Stoikov, and R. Talreja, "A stochastic model for order book dynamics," Operations Research, vol. 58, no. 3, pp. 549–563, 2010.*
2 *L. Gai and S. Yao, "Architecting ultra-low latency platforms via in-memory systems," IEEE Transactions on Services Computing, vol. 16, no. 2, pp. 1142–1155, 2023.*
3 *X. Zhang, H. Yang, and J. Liu, "High-throughput key-value lookup architectures for financial systems," ACM Transactions on Embedded Systems, vol. 21, no. 4, pp. 42–58, 2022.*